

SageApple *Apple II-inspired retrocomputer emulator* Written
in SageLang

Comprehensive Documentation Generated from source
code repository September 8, 2026

Contents

Project Overview	3
Core Components	4
Key Features	4
Applesoft BASIC (~2180 lines)	4
DOS 3.3 (~748 lines)	5
Apple II Monitor (~816 lines)	5
SAGEFS v2 (~371 lines)	5
Test Coverage: 15 Suites, 266 Checks (All Passing)	5
Apps Catalog (12 Classic Apps)	5
Performance Optimizations	6
AVR Port	6
Documentation	6
Current Status	6
Topology	7
Running	7
Commands	7
Stale screen cleanup	8
Note on <code>sys.exec</code> and SSH	8
System layers	8
Component documentation	9
Key design decisions	9
Execution models	10
Memory map (canonical)	10
The 6502-centered pieces at a glance	11
Hardware footprint (MCU budget)	12
How the machine maps to the chip	12
SRAM budget juggling	12
Build, flash, run	12
Regenerating the ROM and the opcode table	13
The host equivalence test	13
Known AVR-specific fixes (worth remembering)	13
Design	14
Tokenizer	14
Expression parser (recursive descent)	14
Statements	14
Errors	15
Program store & persistence shape	15
Test coverage	16
<code>bus.sage</code> — the flat bus	16
<code>AppleBus</code> — the canonical memory map	16
Why RAM is 2 KB on host, 1 KB on the AVR	17
Testing	17
<code>asm6502.sage</code> — two-pass assembler	17
<code>backend.sage</code> — BASIC to 6502	18

Memory map of compiled programs	18
Compilation flow	18
Statement generators	18
Emitted runtime (6502 assembly)	19
Test coverage	19
UART (<code>devices/uart.sage</code>)	19
SPI master (<code>devices/spi.sage</code>)	19
OLED display (<code>devices/display.sage</code>)	20
NOR flash (<code>devices/flash.sage</code>)	20
Speaker (<code>speaker.sage</code>)	20
Test coverage	21
<code>machine.sage</code> — everything in one box	21
<code>boot.sage</code> — the boot ROM	21
<code>echo.sage</code> / <code>terminal.sage</code> — proof and bridge	22
<code>monitor.sage</code> — two monitors	22
<code>os.sage</code> — the console OS	23
<code>dos.sage</code> — the DOS 3.3 command processor	23
<code>storage.sage</code> — SAGEFS-6502	23
<code>graphics.sage</code> — drawing on the OLED	24
<code>apps/catalog.sage</code> — the app catalog	24
Test coverage	24
The CPU class	25
Instruction set	26
Execution	27
Registers (<code>registers.sage</code>)	27
Flags (<code>flags.sage</code>)	27
Ports & uses	27
The suites	28
How suites assert	29
The oracle / equivalence model	29
Running everything	30
<code>avr_boot.sage</code>	30
<code>rom_builder.sage</code>	30
<code>rom_gen.sage</code>	30
<code>gen_table.sage</code>	31
<code>hex_dump.sage</code>	31
<code>applecon.sage</code>	31

title: “SageApple: Apple II-Emulation on ATmega328P” author: “SageApple Project” date: ” oday” — # SageApple Codebase Analysis

Project Overview

Apple II-inspired retrocomputer emulator in **SageLang**, targeting ATmega328P/Arduino UNO R3. - Reusable Sage6502 CPU core (151 opcodes, cycle-exact) - DOS 3.3 command processor - Full Applesoft BASIC interpreter

- Apple II monitor (* prompt) - SAGEFS v2 filesystem - 12 classic apps

Core Components

Component	File	Lines	Purpose
CPU Core	sage6502/cpu.sage	300	Table-driven NMOS 6502, 151 opcodes
Registers	sage6502/registers.sage	60	A/X/Y/PC/SP
Flags	sage6502/flags.sage	95	P register (NV-BDIZC)
Bus	bus/applebus.sage	100	Memory map: RAM/ROM/devices
Devices	devices/*.sage	300	UART, SPI, OLED, flash, speaker
BASIC	basic/basic.sage	2180	Full Applesoft
DOS 3.3	sageapple/dos.sage	54	Commands, file types, errors
Monitor	sageapple/monitor.sage	800	* prompt, dump/disasm/go
OS Shell	sageapple/os.sage	30	1/* unified shell
Filesystem	sageapple/storage.sage	30	SAGEFS v2, 20-byte entries
Compiler	compiler/*.sage	200	asm6502 + BASIC→6502 backend

Key Features

Applesoft BASIC (~2180 lines)

- FP math with scientific notation, 16 functions (SQR, LOG, SIN, etc.)
- String functions (ASC, CHR, *LEFT*, RIGHT, *MID*, LEN, STR\$, VAL)
- Control: IF/THEN/GOTO, FOR/NEXT, GOSUB/RETURN, ON GOTO/GOSUB
- Data: READ/DATA/RESTORE, DEF FN, DIM arrays
- Error handling: ONERR GOTO/RESUME, Applesoft messages
- Token caching, immediate mode RUN/GOTO promotion

DOS 3.3 (~748 lines)

- CATALOG with lock markers (*A 002 NAME), zero-padded sectors
- File types: A/B/T/I
- BSAVE/BLOAD/BRUN with A<addr>,L<len>
- RENAME, LOCK/UNLOCK, MAXFILES 1-16
- MON/NOMON, PR#/IN#
- Error messages: FILE NOT FOUND, TYPE MISMATCH, DIRECTORY FULL vs DISK FULL

Apple II Monitor (~816 lines)

- * prompt via CALL -151/CALL 65449
- Dump (800.), disassembly (800L), go (800G), JSR (800J)
- Store (800:AA BB), step (S), trace (T), display modes (N/I/F)
- E exits to BASIC

SAGEFS v2 (~371 lines)

- 16 entries × 20 bytes (name 12B, size u16, start u16, type, flags)
- Directory: block 0 + 16B of block 1, data from block 2
- First-fit contiguous, format v0x02
- save_applesoft (CRLF), delete erases data blocks

Test Coverage: 15 Suites, 266 Checks (All Passing)

Suite	Checks
6502 CPU/OpCodes	30
BASIC	49
Boot (boot/uart/monitor)	24
Machine (OS/Apple2/Speaker)	69
Compiler (asm6502/backend)	31
Display (SPI/OLED)	38
Storage (Flash/FS)	45

AVR host-equivalence verified

Apps Catalog (12 Classic Apps)

HELLO, COUNTER, BEEP, MASTERCRT, FID, COPYA, DISKRECOV, APPLESOFT, COLOR.DEMO, SHAPE.DEMO, MUSIC.DEMO, HELLO.WORLD + MACHINE1 (6502 binary)

All install via `catalog.install_basic_apps()`, run with `RUN <NAME>`

Performance Optimizations

Optimization	Location	Impact
<code>upd_nz</code> inline bit ops	<code>flags.sage:88</code>	~19% faster
<code>fetch()</code> scratch buffer	<code>cpu.sage:329</code>	No list alloc/instruction
UART <code>tx_text</code> watermark	<code>uart.sage:56</code>	$O(n^2) \rightarrow O(n)$
Bus ROM-first read8	<code>applebus.sage:48</code>	Hot path first
Monitor cached <code>_OPCODES</code>	<code>monitor.sage:669</code>	40× rebuilds eliminated
BASIC token caching	<code>basic.sage:347</code>	Re-tokenize on edit only

AVR Port

`avr/` — C port with PROGMEM tables, generated by `tools/gen_table.sage` and `tools/rom_gen.sage`. Oracle transcript in `host_expected.txt` verified by `make host-test`.

Documentation

`README.md` (images: `SageAppleInfo.png` top, `SageApple.png` in Architecture),
`docs/*.md`, `CHANGELOG.md`, `PLAN.md`

Current Status

All 15 suites, 266 checks passing

AVR host-equivalence verified

12 classic apps functional

README updated with single images

CHANGELOG.md with M14 entry # AppleCon — SageApple board controller

AppleCon is a host-side tool built in SageLang that drives a rack of SageApple boards from a single terminal. It opens with an artistic TUI animation, then drops into a `sage>` shell whose `con N` commands hand off to each board's interactive serial shell.

Topology

All three boards are wired to the OrangePi (192.168.254.44) and reached over SSH:

con	board	serial port	USB controller
0	og Uno R3	/dev/ttyUSB0	mv-ehci (USB4)
1	Nano R3	/dev/ttyUSB1	xhci-hcd (USB2)
2	2nd Uno R3	/dev/ttyUSB2	xhci-hcd (USB2)

Note: The 2nd Uno R3 uses a FIREPHX USB SER (0843:5740) chip that requires the `cdc_acm` kernel module. If the module is not available, `con 2` will report the port as down.

Running

Use the C build of Sage (the self-hosted RISC-V `sage` interpreter is unstable with the animation loop):

```
sage-c tools/applecon.sage
```

Run it from the repo root so `import io/import sys` resolve.

Commands

command	result
<code>sage> con 0</code>	SSH to the OrangePi and <code>screen</code> /dev/ttyUSB0 9600 (og Uno R3)
<code>sage> con 1</code>	SSH to the OrangePi and <code>screen</code> /dev/ttyUSB1 9600 (Nano R3)
<code>sage> con 2</code>	SSH to the OrangePi and <code>screen</code> /dev/ttyUSB2 9600 (2nd Uno R3)
<code>sage> status</code>	probe each port and show which boards are present
<code>sage> help</code>	list the available commands
<code>sage> exit</code>	leave AppleCon

Each `con` connection drops you into the board's boot banner / monitor (`MON>`) or BASIC (1) prompt. `screen` control keys let you return to AppleCon without pulling the board's serial line:

- `C-a d` — detach (leave the connection in the background)
- `C-a k` — kill the screen session and return to the `sage>` prompt
- `C-a ?` — show the screen help

Stale screen cleanup

AppleCon automatically kills any stale detached **screen** sessions on the target port before connecting. This prevents the “Device or resource busy” error that occurs when a previous screen session holds the port open.

Note on `sys.exec` and SSH

Sage’s `sys.exec/sys.shell_exec` security validator only allows alphanumeric, `/ . - _ ~` and spaces — the `@` in `user@host` is rejected. AppleCon therefore writes the `ssh` invocation into a small `/tmp/*.sh` helper (via `io.writebytes`) and `sys.exec("/bin/sh <file>")`, so remote connections work without relaxing the validator.

The remote probe (`status`) uses the same helper technique with `sys.shell_exec` (redirecting stdin from `/dev/null` to prevent the SSH process from consuming piped input) to test whether each remote serial node exists. # Architecture

SageApple is an Apple II-inspired retrocomputer whose software is written almost entirely in **SageLang**, a self-hosted language with its own interpreter (the `sage` binary). The interesting part is what happens at the edges:

- a **table-driven NMOS 6502 CPU emulator** written in SageLang (`sage6502/`),
- a **SageLang-to-6502 compiler backend** (`compiler/`),
- a **SageLang-implemented computer system** layered on top of that CPU (bus, devices, BASIC, monitor, OS, filesystem),
- a **hardware port** of the whole thing to an **ATmega328P (Arduino UNO R3-compatible)** board, where the 6502 core is a faithful C port (`avr/sage6502.c`) run by a thin AVR runtime.

The result is a system that can be booted either by the host `sage` interpreter (fully emulated, used by the test suites) or on the real chip (serial terminal, monitor, BASIC — proven working on silicon).

System layers

+-----+-----+	
apps/ catalog	installed apps: HELLO,
	COUNTER, BEEP, MACHINE1
+-----+-----+	
sageapple/	OS console, SAGEFS
	filesystem, DOS 3.3,
	graphics, monitor,
	boot/terminal
+-----+-----+	
basic/	Applesoft BASIC interpreter
+-----+-----+	

compiler/	asm6502 two-pass assembler
	+ BASIC -> 6502 backend
+-----+	+-----+
bus/ devices/	AppleBus memory map, UART,
	SPI, OLED, NOR flash, spkr
+-----+	+-----+
sage6502/	table-driven NMOS 6502 core
+-----+	+-----+
avr/	C runtime + C port of the
	core for the ATmega328P
+-----+	+-----+

Every layer above `bus/` executes on top of the 6502 core. The core never touches host memory directly: all memory access is routed through a `Bus` interface, so the very same CPU runs inside the host interpreter and on the AVR chip.

Component documentation

Document	Covers
docs/sage6502.md	CPU core, registers, flags, instruction set, timing
docs/bus.md	flat bus, AppleBus memory map, I/O ports
docs/devices.md	UART, SPI, OLED display, NOR flash, speaker
docs/basic.md	Applesoft BASIC interpreter
docs/compiler.md	6502 assembler + SageLang-to-6502 backend
docs/os.md	OS console, monitor, filesystem, graphics, boot
docs/avr.md	ATmega328P port, C runtime, flashing
docs/tools.md	host-side tools (hex, ROM, table generation)
docs/tests.md	the 15 validation suites

Key design decisions

1. **The CPU never knows about the host.** All reads/writes go through the `Bus`/`AppleBus` interface. This is what lets the identical core run under host emulation and on the AVR with a C bus backed by SRAM + flash PROGMEM.
2. **BASIC runs natively as SageLang**, not as 6502 code. The interpreter lives in `basic/basic.sage` and drives the 6502 machine peripherally; for standalone execution on the chip the host side interprets BASIC while

the system runs under the C port of the CPU. The `compiler/backend` exists to *compile* BASIC programs to 6502 for the cases where native 6502 execution matters.

3. **Framebuffer lives on the display controller** (1024 bytes, page-major), never in the 2 KB of SRAM.
4. **Static allocation and bit packing everywhere.** No heap, no dynamic resources; data structures are pre-sized to fit the AVR budget.
5. **The same session is the test.** `tools/rom_gen.sage` runs a canonical terminal session (help, regs, peek/poke, dump, reset...) and records the byte-exact transcript into `avr/host_expected.txt`. The AVR port replays the identical session through the C core (`make host-test`) and must produce byte-identical output — host bus vs AVR bus equivalence, proven mechanically.

Execution models

Model	Where	How
Host (emulated)	<code>sage</code> interpreter on the host	<code>sageapple/machine.sage</code> composes <code>AppleBus</code> + CPU; tests drive it directly
Host (hard 6502)	compiled C core on the host	<code>avr/</code> built with <code>-DHOST</code> runs <code>sage6502.c</code> natively for equivalence tests
Device (silicon)	ATmega328P	<code>make flash</code> burns the same core + monitor ROM; the 6502 talks to a real UART

Memory map (canonical)

Range	Device
\$0000-\$07FF	2 KB RAM
\$2000-\$2001	UART status/data
\$2002-\$2004	SPI display control
\$2005-\$2006	SPI flash control
\$2007	speaker tone (0 = silence)
\$3000	legacy console TX alias
\$8000-\$FFFF	32 KB program ROM (read-only)

(On the AVR target RAM is truncated to \$0000-\$03FF — 1 KB, the second KB of the host budget belongs to the emulator state; see docs/avr.md.)

The 6502-centered pieces at a glance

module	purpose
sage6502/cpu.sage	table-driven NMOS 6502 interpreter, 151 canonical opcodes, all 13 addressing modes, cycle counting, interrupts
sage6502/registers.sage	A/X/Y/SP/PC object
sage6502/flags.sage	processor status byte NV-BDIZC
sage6502/constants.sage	named instruction IDs and addressing modes
sage6502/cycles.sage	256-entry base cycle-count table
sage6502/decimal.sage	pure ADC/SBC decimal helper functions
sage6502/opcodes.sage	declarative opcode table (parallel to cpu.sage)
bus/bus.sage	64 KB flat byte-array bus
bus/applebus.sage	the AppleBus: RAM + ROM + memory-mapped devices
devices/*.sage	UART, SPI master, OLED display, NOR flash, speaker models
basic/basic.sage	Applesoft BASIC interpreter (PRINT/LET/GOTO/IF/FOR/INPUT/GOSUB/DEF FN/READ/DATA...)
compiler/asm6502.sage	two-pass 6502 assembler
compiler/backend.sage	BASIC -> 6502 compiler with runtime
sageapple/*.sage	OS, monitor, storage/SAGEFS, graphics, machine, terminal, echo, boot
apps/catalog.sage	preinstalled app catalog
avr/*	C runtime, C port of the core, linker/startup, make/avrdude

See PLAN.md for the original plan, phases, and design notes, and CHANGELOG.md for the milestone log. # The AVR port — avr/

The hardware milestone (M13): the same SageApple 6502 emulation, running on a real **ATmega328P** / **Arduino UNO R3-compatible** board. The 6502 core is ported to C (avr/sage6502.c) and driven by a tiny C/ASM runtime (avr/main.c, avr/start.S, avr/avr.ld).

```
avr/
  Makefile      build sageapple.hex, flash, fuse, host-test targets
  avr.ld        ATmega328P linker script (32K flash / 2K SRAM / 1K EEPROM)
```

```

start.S      reset → stack init → .bss clear → .data copy → main()
main.c       UART init (9600 8N1) + emulator main loop
sage6502.c   C port of the 6502 core (table-driven, PROGMEM OP table)
bus.c        1 KB RAM + UART/ROM device window on the MCU
rom_avr.c    include of the generated monitor ROM (PROGMEM)
rom.inc      generated: const MONROM[8192] (monitor at $E000-$FFFF)
rom_host.c   same ROM as a plain array for -DHOST builds
host_main.c  host equivalence driver (replays the oracle session)
host_uart.c  host UART stub (stdio)
host_cmds.txt the canonical session commands
host_expected.txt the oracle transcript (byte-exact expected output)

```

Hardware footprint (MCU budget)

resource	used
flash	~12.3 KB of 32 KB (includes 8 KB monitor ROM)
SRAM	1,036 B of 2,048 B (1 KB 6502 RAM + emulator state)
UART	USART0, 9600 8N1, 16 MHz
clock	16 MHz (external crystal, CKDIV off: lfuse 0xFF)

How the machine maps to the chip

6502 addr	AVR mapping
\$0000-\$03FF	1 KB <code>ram[]</code> in SRAM (see note below)
\$2000	UCSR0A bit0 = RX-ready, bit1 = TX-ready
\$2001	UDR0 (RX read / TX write)
\$E000-\$FFFF	MONROM[8192] in flash PROGMEM (via <code>pgm_read_byte</code>)

Everything else reads 0xFF and ignores writes.

SRAM budget juggling

The host model gives the machine 2 KB RAM; the 328P has 2 KB SRAM total. The 1 KB \$0000-\$03FF mapping leaves the rest for the emulator (registers, stacks, buffers). This is intentional (see `docs/bus.md`).

Build, flash, run

```
sudo apt-get install gcc-avr avr-libc binutils-avr avrdude
```

```

cd avr
make                                # -> sageapple.elf / sageapple.hex
make flash                          # avrdude -c arduino -P /dev/ttyUSB0 -b 115200
# if the chip is fuse-locked to an external clock, use a USBasp first:
avrdude -p atmega328p -c usbasp -B 3 -U lfuse:w:0xFF:m -U hfuse:w:0xD9:m -U efuse:w:0xFF:m

```

Terminal (the monitor is 9600 8N1):

```
screen /dev/ttyUSB0 9600          # or picocom -b 9600 /dev/ttyUSB0
```

On the board you get the real monitor session:

```

SageApple MonitorMON> help
Commands: help dump peek poke regs run reset
MON> regs
A=00 X=FD Y=11 SP=FD P=68
MON> peek 0000
0000: 42
...

```

Regenerating the ROM and the opcode table

```

sage ../tools/rom_gen.sage          # rebuild avr/rom.inc + host_expected.txt
sage ../tools/gen_table.sage        # regenerate OP[256] in sage6502.c from the
                                   # canonical Sage_OPCODES table

```

Why gen_table exists: sage6502.c used to carry a hand-maintained opcode table that drifted from the core. tools/gen_table.sage regenerates it straight from sage6502/cpu.sage's _OPCODES, so the C port can never drift from the canonical core.

The host equivalence test

```
make host-test
```

Builds the *same* sage6502.c + bus.c for the host (-DHOST), replays the exact oracle session (host_cmds.txt, 60k steps per burst), and diffs against avr/host_expected.txt. This is the mechanical guarantee that the C port behaves like the SageLang machine.

Known AVR-specific fixes (worth remembering)

1. **Stack pointer init** — start.S must load Z before the .bss clear loop (st Z+ was previously writing through a zeroed Z into the register file/SP). Fixed.
2. **Opcode table in flash** — static const uint16_t OP[256] must be declared PROGMEM and read with pgm_read_word; a plain ld on AVR reads SRAM, not flash, silently corrupting the dispatch (the host build is unaffected).

3. **SRAM size** — the 2 KB RAM model physically cannot coexist with the emulator; use the \$0000-\$03FF model.
4. **BRK** — the core halts the machine on BRK (a documented deviation; the monitor's `run` reports “no user program” instead of crashing).# Applesoft BASIC — `basic/basic.sage`

A full Applesoft-compatible BASIC interpreter written in SageLang (~2160 lines). It runs **natively** (as Sage code, not 6502 machine code) and drives the machine peripherally — the same design is used by the DOS 3.3 command processor and the host test harness.

Design

- numbered program store: `prog` is a *sorted* list of `[line_number, text]`;
- `vars` — real-number variables (`a..z`, `a0..a9` and `a$..z$` string variants);
- INPUT works from either a queue (`inq`, scripted by the harness) or interactive `feed(ch)` streaming (the terminal path);
- output accumulates in `out` — the OS/the tests read it via `out()`.

Tokenizer

`tokenize(line)` normalizes identifiers to upper case and produces a token stream: `["num", n]`, `["str", s]`, `["id", WORD]`, `["sym", c]`, `["o", ...]`. Numbers include scientific notation (1E9), and string literals use `"..."` (doubled `"` inside for embedded quotes).

Expression parser (recursive descent)

```
eval_arith -> eval_add ( + - ) with < > via eval_mul
eval_mul   * / ^ (divide-by-zero -> DIVISION BY ZERO;
             ^ overflow checked before computing)
eval_un    unary minus
eval_atom  numbers, ( ... ), variables, array subscripts, functions
```

Floating-point arithmetic; INT, SQR, ABS, SGN, RND, EXP, LOG, SIN, COS, TAN, ATN, FRE, PEEK, PDL, POS, SPC, TAB are numeric; ASC, CHR\$, LEFT\$, RIGHT\$, MID\$, LEN, STR\$, VAL are string functions. `&` is not an operator — it triggers the in-line CALL-style handling for `& + hex` (Apple II monitor call).

Statements

statement	behavior
PRINT	strings/expressions, ; and , separators, TAB/SPC, trailing ;/, suppresses CRLF

statement	behavior
LET x=expr / x=expr	default assignment when the first token is an identifier
GOTO n	jump by line number
IF cond THEN stmts / IF cond	conditions = < <= > >= <>, bare
GOTO n	nonzero = true
FOR v=a TO b [STEP s]	push frame [name, limit, step, line_index]
NEXT v	apply step, reloop while the test holds
INPUT [prompt;]var[,...]	take from the queue or input prompt
GOSUB n / RETURN	subroutine call/return
ON expr GOTO n1,n2,... / ON expr	computed branch
GOSUB ...	
READ v[,...] / DATA ... / RESTORE	data statements
DEF FN name(x)=expr	user functions (FN name(...))
DIM name(n)	arrays (0-based, bounds-checked: BAD SUBSCRIPT)
GET v	single-character input
POKE addr,val / CALL addr	drive the machine bus (CALL -151/CALL 65449 enter the monitor)
BEEP [freq]	optional speaker tone
HOME	clear the terminal (ANSI ESC[2J ESC[H)
ONERR GOTO n / RESUME	error trapping
END / STOP	stop the program
REM	skip comment

Program editing: LIST, NEW, RUN (with RUN n form), CONT; a 1,000,000-step guard prevents runaway loops; statements may be separated by : on one line; every program ends silently (no OK), idle prompt].

Errors

Reports match Applesoft shape: ?SYNTAX ERROR IN n, ?TYPE MISMATCH ERROR IN n, DIVISION BY ZERO, ?ILLEGAL QUANTITY ERROR, OVERFLOW, ?UNDEF'D STATEMENT ERROR IN n, ?NEXT WITHOUT FOR ERROR IN n, BAD SUBSCRIPT, OUT OF DATA, REDIM'D ARRAY, UNDEF'D FUNCTION. All are trap-pable with ONERR GOTO (the error code is available for RESUME/conditional handling).

Program store & persistence shape

- set_line(n, text) — sorted insert, or replace same-number lines;
- find_line(n) returns content index;

- the OS serializes programs via DOS (`SAVE file`, CRLF-joined lines, preserving exact list order — see docs/os.md).

Test coverage

- `tests/basic/test_basic.sage` — 49 checks (Applesoft expressions, control flow, errors, string functions)
- `tests/machine/test_apple2.sage` — 30 checks (DOS verbs, file types, monitor shell, `CALL -151`, `POKE/PEEK` interop) # The bus layer — `bus/`

Two modules define how the 6502 sees its memory:

```
bus/
  bus.sage          flat 64 KB byte-array bus (generic, tests)
  applebus.sage     the real SageApple memory map + devices
```

`bus.sage` — the flat bus

Bus wraps a single 65,536-byte array:

```
read8(addr)        masked 8-bit read
write8(a, v)        masked 8-bit write
read16(addr)        little-endian
load(image, base)   bulk-load a byte list at a base address
```

This is the minimal interface the CPU core needs. `AppleBus` (below) subclasses the same contract with memory-mapped devices — the CPU does not care which one it is attached to, which is what keeps the core portable to the AVR.

`AppleBus` — the canonical memory map

`applebus.sage` instantiates the device set and owns:

- `ram` — 2048 bytes (host) at `$0000-$07FF`
- `rom` — 32768 bytes at `$8000-$FFFF`
- a `UART()`, `DisplaySPI()`, `Flash()` chip, `FlashSPI()` controller, `Storage()` filesystem, `Speaker()`

read/write dispatch:

address	read	write
<code>\$0000-\$07FF</code>	RAM	RAM (also <code>write_ram()</code>)
<code>\$2000</code>	UART status (bit0 RX-ready, bit1 TX-ready)	—
<code>\$2001</code>	UART RX data	UART TX data
<code>\$2002</code>	—	display command (DC low)
<code>\$2003</code>	—	display data (DC high)
<code>\$2004</code>	display status (always <code>\$80</code>)	display reset
<code>\$2005</code>	flash response byte	flash byte transfer

address	read	write
\$2006	flash CS level	flash CS level set
\$2007	\$00	speaker tone (0 = silence)
\$3000	—	legacy console TX alias
\$8000-\$FFFF	ROM (loads via <code>load_rom()</code>)	dropped (read-only)
anything else	\$00	ignored

Convenience paths used by the host side: `read16()`, `load_rom(image)`, `console_output()` → `uart.tx_text()`.

Why RAM is 2 KB on host, 1 KB on the AVR

The host emulator gives the machine a full 2 KB \$0000-\$07FF. The AVR target has only 2 KB of physical SRAM: the emulator state (registers, the C stack, trace buffers) must share it with the 6502's own RAM, so `avr/bus.c` maps \$0000-\$03FF (1 KB) — the monitor, BASIC workspace and current programs fit comfortably. See `docs/avr.md`.

Testing

`tests/boot/*` and every device test drive the bus; the OS tests (`tests/machine/test_os.sage`) boot a full AppleBus system end-to-end. # Compiler & assembler — `compiler/`

Two tools turn 6502 assembly and (a subset of) BASIC into 6502 machine code:

```
compiler/
  asm6502.sage      two-pass 6502 assembler
  backend.sage      BASIC -> 6502 compiler front-end + runtime
```

asm6502.sage — two-pass assembler

A 139-entry opcode map (`fadd(mnemonic, mode, opcode)`) indexes `mnemonic+mode` pairs to bytes. Supported instruction classes:

- loads/stores: LDA/LDX/LDY, STA/STX/STY with the standard memory modes (immediate, zero page $\pm X/Y$, absolute $\pm X/Y$, (`zp,X`) and (`zp`), `Y` indirect-indexed for LDA/STA);
- ADC/SBC/AND/ORR/EOR/CMP: immediate/`zp/zpX/abs/absX/absY` (the indirect-indexed forms are not emitted by the assembler);
- BIT, INC/DEC, shifts/rotates (accumulator + memory forms), JMP (abs + indirect), JSR, RTS/RTI, the eight branches, flags, stack ops, NOP, BRK.

Language features:

- `org <addr>` and `.byte v1, v2, ...` pseudo-ops;

- labels: **name**: (whitespace-free), forward references resolved by a **two-pass** size-then-emit strategy;
- **#<label** and **#>label** low/high immediates;
- **(zp),Y / (zp,X) / (abs)** addressing; **\$nn/\$nnnn** auto-select zero page vs absolute;
- relative branch offsets computed as **(target - (start + 2)) & 0xFF**;
- comments after **;**.

Entry point: `asm(source_lines, base) -> [image, labels]`; `hexb(v)` formats bytes. It is used by the monitor ROM builder, the OS 6502 app runner, the app catalog, and all compiler tests.

backend.sage — BASIC to 6502

A narrowing compiler: it compiles a *numbered BASIC program* (the PRINT/LET/GOTO/IF subset; no FOR/NEXT, no INPUT, no user functions) to standalone 6502 code.

Memory map of compiled programs

area	purpose
\$20-\$3E	variables A..Z (26 zero-page bytes)
\$7C	expression temp (left operand / lvalue)
\$7E	expression temp (right operand / comparison)
\$7D, \$7F	decimal-print scratch
\$10/\$11	string pointer
\$1A-\$1C	multiply scratch
\$0300+ (typical)	program + runtime (assembled at caller-chosen base)

Compilation flow

1. lines de-duplicated by number (later wins), sorted numerically;
2. `org <base>`, a label `L<n>` per line, statement code;
3. `HALT: JMP HALT`, the runtime, then string data as `.byte` lists (`S0: .byte 48,45,...`).
4. output assembled by `asm6502.asm()`; returns `[image, halt_pc, labels]`.

Statement generators

- `gen_print` — strings (into `S#` labels), expressions, blank lines, `;` suppresses CRLF;

- `gen_let` — evaluates `expr`, `STA $20+X`;
- `gen_goto` — `JMP L#`; unknown target prints ? then halts;
- `gen_if` — `THEN / GOTO` variants, condition skips/jumps;
- `gen_cmp` — `$7C/$7E` temps, `SEC`; `SBC` compare;
- `gen_expr` — numbers `LDA #`, variables `LDA $zp`, negation via `EOR #$FF`; `CLC`; `ADC #$01`, `+` `-` `*` `/` via runtime.

Emitted runtime (6502 assembly)

`R_OUT/R_OUTW` (UART busy-wait `$2000/$2001`), `R_PSTR` (zero-terminated string at `(0x10),Y`), `R_PRINT2` (up-to-3-digit decimal), `R_CRLF`, `R_MUL` (8-iteration shift-add), `R_DIV` (repeated subtraction; divisor 0 yields 0).

Test coverage

- `tests/compiler/test_asm6502.sage` — 11 checks (encoding, labels, emulator execution)
- `tests/compiler/test_backend.sage` — 20 checks (arithmetic, strings, `GOTO/IF` loops, comparisons, divide-by-zero).# Devices — `devices/`

The five memory-mapped devices of SageApple, modeled in SageLang. All live behind the AppleBus I/O window `$2000-$2007` plus the flash controller at `$2005-$2006`.

`devices/`

<code>uart.sage</code>	UART terminal (RX FIFO + TX transcript)
<code>spi.sage</code>	bit-level SPI master with attachable slave
<code>display.sage</code>	SSD1306-style 128x64 OLED controller
<code>flash.sage</code>	SPI NOR flash chip + bus-side controller
<code>speaker.sage</code>	PWM speaker / tone generator

UART (`devices/uart.sage`)

- `RX: receive(ch) / receive_str(s)` push bytes into an RX FIFO (the keyboard side).
- `status()` — `bit0` = RX-ready, `bit1` = TX-ready (always set), i.e. `0x02` normally, `0x03` when input is waiting.
- `tx_write(ch)` appends to a TX byte list; `rx_read()` pops the FIFO (`0x00` when empty).
- Host readback: `tx_len()`, `tx_text()` (renders the TX list, translating `\r/\n`), and a printable-ASCII filter via the module-local `_printable()`.

On the AVR the same port address drives the *real* MCU UART (see `docs/avr.md`).

SPI master (`devices/spi.sage`)

Bit-level bit-banged master:

```
transfer8(byte)      clocks 8 bits MSB-first on MOSI, samples MISO,
                     calls slave.spi_frame(master, byte) at frame end
cs_low()/cs_high()  active-low chip select, calls slave.spi_cs(level)
```

Slave duck-typing: `spi_cs`, `spi_clk(master, bit_out)`, `spi_frame(master, byte)`. `transfers` counts frames (assertable in tests). The AVR build drives the SPCR/SPDR registers instead of this bit model — same protocol.

OLED display (`devices/display.sage`)

SSD1306-style 128x64 mono controller as a SPI slave:

- 1024-byte page-major framebuffer (8 pages $\times$ 128 columns) — the framebuffer *lives on the controller*, never in the 6502's RAM;
- byte-level protocol decode: column low/high nibbles (\$00-\$1F)
- page set \$B0-\$B7, display on/off \$AE/\$AF, memory mode (\$20), column/page windows (\$21, \$22 with params), contrast (\$81), charge pump (\$8D), all params absorbed correctly;
- data bytes land at `fb[page*128+col]` with auto-increment and window wrap in non-page mode.

Default window is full 0..127 columns $\times$ 0..7 pages. `DisplaySPI` is the bus-facing shell: `cmd()` (DC low), `dat()` (DC high), `status() == 0x80`, `reset()` re-arms the controller.

NOR flash (`devices/flash.sage`)

Models a 64 KB SPI NOR flash, JEDEC id EF 40 15 (Winbond-family):

- \$06/\$04 write-enable latch set/clear; \$05 status register (bit1 = WEL);
- \$03 read (3-byte address stream, then data stream);
- \$02 page program (requires WEL);
- \$20 4K sector erase (requires WEL) — address masked to `addr & 0xFF000`;
- addresses masked with `addr & (size-1)`; erased state is `0xFF`;
- `FlashSPI(gpu, chip)` is the bus controller: `byte(b)` executes a transfer; `csl(level)` raises/asserts CS.

Speaker (`speaker.sage`)

- `tone(freq, duration)` — records a tone (only `freq > 0`, 0 silences and is not recorded);
- `silence()`, `beep()` (= `tone(1000, 100)`), `count()`, `tone_at(i)`, `last()`.

The AVR would drive a piezo via a TIMER; the host model keeps a transcript that tests assert against.

Test coverage

- `tests/display/test_spi.sage` — 9 checks (frame protocol, CS, counters).
- `tests/display/test_display.sage` — 29 checks (decode, addressing windows, pixels, lines, glyphs, 6502-driven \$2002/\$2003 access).
- `tests/storage/test_flash.sage` — 19 checks (power-on erase, JEDEC, WEL/protection, page program/read, 4K erase, 6502-driven \$2005/\$2006).
- `tests/machine/test_speaker.sage` — 12 checks (device, BASIC BEEP, 6502 STA \$2007). # Applications & OS — `sageapple/`, `apps/`

The OS layer: everything that turns the 6502 + bus into a usable SageApple computer — boot, monitor, console OS, filesystem, graphics, and the installed app catalog.

```
sageapple/
  machine.sage      composition: AppleBus + CPU + power_on()
  boot.sage         boot ROM builder (banner + reset vector)
  echo.sage         UART echo ROM (host <-> 6502 proof)
  terminal.sage     host terminal bridge
  monitor.sage      AVR monitor ROM ($E000) + host Apple II monitor
  os.sage           the console OS (`]`/*` shell)
  dos.sage          DOS 3.3 command processor (CATALOG/SAVE/LOAD/...)
  storage.sage      SAGEFS-6502 filesystem
  graphics.sage     5x7 font + drawing primitives
  apps/catalog.sage installed apps (HELLO/COUNTER/BEEP/MACHINE1)
```

`machine.sage` — everything in one box

```
SageApple
  bus    AppleBus (ram, UART, display, flash, storage, speaker)
  cpu    CPU(bus)
power_on()      -> load boot ROM, cpu.reset(), cpu.run()
boot_rom(image) -> load an arbitrary ROM + reset
uart_receive(s) -> queue terminal input
console_text()  -> terminal back-buffer (tx_text)
uart_status() / tx_len() / tx_text()
```

`power_on()` is how host demos and tests start the machine.

`boot.sage` — the boot ROM

A 32 KB image holding:

- at \$8000 — a small program: walk a string with `LDX #0; LDA $8040,X; STA $2001; INX, CSE`; when 0 is reached, loop forever;
- at \$8040 — the classic banner:

SageApple Computer
Sage6502 CPU ... OK
OUTPUT ...

- reset vector \$FFFC/\$FFFD = 00 80 → boot at \$8000.

echo.sage / terminal.sage — proof and bridge

`build_echo_rom()` is an 8-byte poll loop: LD \$2000; AND #1; BEQ ...; LDA \$2001; STA \$2001; JMP Terminal wraps a SageApple running that ROM, `feed(s)` queues input, and `pump()` steps the CPU until stable and returns the *increment* of the TX text — the model for the host-emulated terminal sessions in the test suite and the transcript oracle.

monitor.sage — two monitors

Two things live in this file:

AVR monitor ROM — written in 6502 assembly and built with `asm6502` inside an 8 KB window, typically loaded at \$E000:

help	command list
dump aaaa bbbb	hexdump range
peek aaaa	byte at address
poke aaaa vv	write byte
regs	show P C X Y SP
run	run the user program (stub)
reset	warm restart

Dispatch reaches long-distance handlers with absolute JMP trampolines (branch range is only ± 127 bytes). The monitor's exact boot + command session is the byte-for-byte “oracle” transcript (`tools/rom_gen.sage` → `avr/host_expected.txt`) that the C port must reproduce (`make host-test` and the on-board terminal).

Host Apple II monitor — the interactive `Monitor` class behind the `*` prompt (entered with `CALL -151` or the `monitor` command), with real Apple II commands:

* 300.30F	forward dump	* 30F-300	backward dump
* 300:41 42	store bytes	* 300G	go
* 300J / 300C	jsr/call	* R	run reset vector
* S	step (T = trace)	* 300L	disassemble
* E	exit to BASIC	* N / I / F	display mode

It drives the CPU directly (steps/executes through `cpu.step()`), so `G/J` run real 6502 code in RAM and `L` disassembles from the opcode table.

os.sage — the console OS

A unified `]/*` shell: DOS 3.3 verbs, Applesoft BASIC at the `]` prompt, and the Apple II monitor at the `*` prompt:

```
help          # OS + DOS + BASIC command summary
info          # machine summary + port map
apps          # catalog, [I] = installed
dir           # = CATALOG (DOS listing)
splash        # OLED: charge pump, mode, flip, offset, on, draw "SageApple"
clear         # clear the terminal (ANSI ESC[2J ESC[H)
basic         # into BASIC ("] ")
monitor       # into the 6502 monitor ("* ")
run <name>    # DOS RUN: run a saved program
exit / quit   # BYE
everything else # DOS verbs (catalog/save/load/delete/rename/lock/
               #   unlock/verify/mon/nomon/pr#/in#/maxfiles/init/
               #   open/close/read/write/append/position/bload/
               #   bsave/brun/exec/fp/int) or an Applesoft line
```

On boot the OS self-checks CPU / memory / ROM / UART / SPI display / flash and prints **SageApple OS 0.1** before the `]` prompt. `RUN <file>` output from the DOS path is drained from the BASIC interpreter too, so programs prompting for input resume correctly over the shell.

dos.sage — the DOS 3.3 command processor

`dos.command(cmd)` parses a DOS verb and drives BASIC/storage:

- CATALOG — real DOS listing (DISK VOLUME 254, A 002 HELLO, *A lock marker, zero-padded 3-digit sectors, sequential sector counts);
- SAVE/LOAD/RUN/VERIFY — Applesoft programs (type A);
- BSAVE/BLOAD/BRUN — binary files with A<addr>/L<len> args (type B); RUN/BRUN position the CPU and execute;
- DELETE/RENAME/LOCK/UNLOCK — directory maintenance;
- MAXFILES n — 1..16, else RANGE ERROR;
- MON/NOMON, PR#n/IN#n, INIT — standard DOS behaviour (PR#/IN# with a slot argument, PR#0/IN#0 treated as one word);
- OPEN/CLOSE/READ/WRITE/APPEND/POSITION/EXEC/FP/INT — accepted and error-checked (file operations map to SAGEFS).

Errors follow DOS conventions: FILE NOT FOUND, FILE TYPE MISMATCH, DIRECTORY FULL vs DISK FULL (an empty slot scan distinguishes them), RANGE ERROR.

storage.sage — SAGEFS-6502

A minimal filesystem over the SPI NOR flash:

constant	value
block size	256 bytes
directory entries	16 (20 bytes each)
data blocks	254 (block 2 .. 255)

Layout:

- block 0 [0..1] magic "SF", byte 2 format version 0x02, then 16 entries of 20 bytes: name (12 bytes NUL-padded), size (u16 LE), start block (u16 LE), type byte, flags byte; the directory spans block 0 plus 16 bytes of block 1, so data begins at block 2 — ~62 KB usable.

File types follow DOS: A Applesoft (0x41), B binary (0x42), T text (0x54), I integer (0x49); flags carry the lock bit.

API: `format()`, `save_blob(name, bytes)` (first-fit contiguous), `load_blob(name)`, `delete(name)` (zeroes the entry), `list()`, `size_of(name)`, `find(name)`, `name_at(i)`, `type_at(i)`, `flags_at(i)`, `file_type(name)`, `sectors_of(name)`, `is_locked(name)`, `lock_file(name)`, `unlock_file(name)`, `rename_file(old, new)`, plus `save_text/load_text` and `save_applesoft` (CRLF-joined lines) for BASIC workspace persistence. A full directory fails cleanly (-1).

graphics.sage — drawing on the OLED

- `_FONT` — packed 5x7 bitmap font for ASCII 0x20..0x7A (91 glyphs, 5 bits per row);
- `Gfx` over the display controller: `clear()`, `put_fb()`, `pixel/get`, Bresenham `line()`, `char()`, `text()` with 6-pixel advance (max 21 chars per line at 128 px).

apps/catalog.sage — the app catalog

- BASIC programs installed into SAGEFS: **HELLO**, **COUNTER** (FOR ... TO 3 ... NEXT), **BEEP** (2000/4000 Hz tones);
- **MACHINE1** — a *compiled* 6502 app (compiler) that prints 6502 APP OK over UART and plays a 60 Hz tone — native 6502 execution proof in the OS.

Test coverage

suite	checks	covers
tests/boot/test_boot.sage	1	boot ROM wiring
tests/boot/test_uart.sage	1	echo / terminal
tests/boot/test_monitor.sage	1	AVR monitor session
tests/machine/test_os.sage	24	the definition of done

suite	checks	covers
tests/machine/test_appl32.sage	132	DOS verbs, file types, monitor shell, CALL -151, POKE/PEEK
tests/machine/test_speaker.sage	12	speaker from BASIC + 6502
tests/storage/*	45	flash + filesystem

`sage6502/` implements a complete, table-driven interpreter for the NMOS 6502 microprocessor, written in pure SageLang. It is the beating heart of the project: the same core (conceptually) runs under the host interpreter and inside the ATmega328P on the real board.

```
sage6502/
  registers.sage  A/X/Y/SP/PC register object
  flags.sage      processor status register (NV-BDIZC)
  constants.sage  named instruction IDs and addressing modes
  opcodes.sage    declarative opcode-to-[id, mode] table
  cycles.sage     256-entry base cycle-count table
  decimal.sage    pure ADC/SBC decimal helper functions
  cpu.sage        table-driven interpreter (uses the modules above)
```

The CPU class

`cpu.sage` defines CPU with a bus reference — *never* host memory:

field	meaning
<code>bus</code>	the memory bus (Bus flat or AppleBus)
<code>regs</code>	<code>Registers</code> object (a, x, y, sp, pc)
<code>status</code>	<code>Status</code> object (the P register)
<code>halted</code>	machine is halted (BRK or user request)
<code>cycles</code>	instruction cycle counter
<code>irq_pending</code> / <code>nmi_pending</code>	soft-latched interrupt requests

Public API:

```
reset()           ; SP=$FD, PC=read16($FFFC), P=B|I, A/X/Y=0
step()            ; fetch+decode+execute one instruction
run()             ; step with a 1,000,000-cycle guard
interrupt() / nmi() ; latch a pending IRQ / NMI
push()/pull()/push16()/pull16()
read8()/write8()/read16()
```

Interrupts are serviced *between* instructions: NMI pushes PC + status, sets the I flag, and vectors to \$FFFA; IRQ to \$FFFE (only when I is clear). Both account +7 cycles. BRK (by design quirk, not real 6502 behavior) sets `halted = true`.

Instruction set

The opcode space is defined by tables built at import time:

table	size	meaning	lives in
<code>_OPCODES</code>	256	[<code>instruction_id</code> , <code>cpu.sage</code> <code>addressing_mode</code>] (<code>_build_opcodes()</code>) per opcode byte	
<code>_CYCLES</code>	256	base cycle count per opcode byte	<code>cpu.sage</code> (<code>_load_cycles()</code>)
<code>_INSN</code>	—	mnemonic strings (used by docu- mentation/tools)	<code>cpu.sage</code>

`constants.sage` provides named IDs (LDA=0, LDX=1, ..., BRK=55) and addressing modes (M_IMM=0, M_ZP=1, ..., M_IMP=12) so dispatch sites read like `id == ADC` instead of magic numbers. The opcode table is built locally in `cpu.sage` via `_build_opcodes()` (kept inside the file to sidestep SageLang module-import scoping); a parallel `opcodes.sage` module exposes the same table for external tools.

There are 56 instruction ids (LDA, LDX, LDY, STA, STX, STY, TAX, TXA, TAY, TYA, TSX, TXS, PHA, PHP, PLA, PLP, ADC, SBC, AND, ORA, EOR, BIT, CMP, CPX, CPY, INC, DEC, INX, INY, DEX, DEY, ASL, LSR, ROL, ROR, the 8 branches, JMP/JSR/RTS, BRK/RTI, NOP, CLC/CLD/CLI/CLV/SEC/SED/SEI) and **13 addressing modes** (see `constants.sage` for the full list).

Notes on fidelity:

- zero-page indexed addressing wraps within the zero page;
- absolute indexed (`abs,X` `abs,Y`) and indirect indexed (`((zp),Y)`) reads charge the +1 cycle page-cross penalty, as does a taken branch whose target crosses a page (base `_CYCLES` counts already include the RMW/store page-cross cycles, matching the NMOS 6502);
- `JMP ($xxFF)` reproduces the NMOS quirk: the high byte is read back on the same page (`$xx00`), not from the next page;
- indirect modes read through zero-page pointers with page-wrap on pointer increment;
- undefined opcodes decode to `[0, M_IMP]` and execute as cycle-free no-ops;
- stores (STA/STX/STY — ids 3/4/5) write without the dummy read, so an RX input port is never consumed by a store.

Execution

- `step()` — fetch opcode at PC, PC++, `decode()`, `exec(id, mode)`, add `_CYCLES[op]` plus the extra cycles returned by `exec()` (+1 for a taken branch, +1 more if the branch target crosses a page, +1 for an indexed read crossing a page), then serve a pending interrupt.
- `exec(id, mode)` — immediate / accumulator (shifts on A) / branch / implied / memory dispatch.
- Arithmetic — `adc()` / `sbc()` use the canonical overflow formula `(a^r)&(operand^r)&0x80`; `compare()` sets C from `reg >= operand`; shifts/rotates set C from the bit shifted out.
- `fetch(mode)` computes the effective address for every non-immediate, non-implied mode.

Registers (`registers.sage`)

Registers holds:

a, x, y - 8-bit
sp - 8-bit (stack page fixed at \$0100)
pc - 16-bit

`get_*/set_*` mutators mask values to the register width (PC 16-bit, the rest 8-bit).

Flags (`flags.sage`)

Single-byte processor status:

7 6 5 4 3 2 1 0 = N V - B D I Z C

Status exposes `N()` `V()` `B()` `D()` `I()` `Z()` `C()` getters, `set_N()`... setters using mask-clear semantics, raw `get()/set()/from_byte()`, and `upd_nz(value)`. Bit 5 is never exposed.

Ports & uses

- host: `bus.applebus` (via `sageapple/machine.sage`), all tests
- the C port in `avr/sage6502.c` mirrors the dispatch exactly — including the opcode→[id, mode] table *and* the base cycle-count table, both *generated* from this file by `tools/gen_table.sage`;
- the assembler and the compiler backend array opcode encodings from the same canonical NMOS numbers.

Known randomness to preserve: BRK halts the machine; undefined opcodes are NOPs; cycle accounting is exact for the page-cross penalties above. # Tests — `tests/`

SageApple is validated by **15 SageLang suites: 266 checks**, each self-contained (Results: N passed, 0 failed + ALL OK on success), each runnable directly:

```
sage tests/6502/test_cpu.sage
sage tests/boot/test_monitor.sage
...
```

The suites

module	checks	what it proves
tests/6502/test_cpu.sage	6	CLI/LDA/TAX/INX/STX/ADC/STA, SBC, branch skip, flags
tests/6502/test_opcodes.sage	21	abs,X / abs,Y addressing, page wrap, stack, JSR/RTS, branches, IRQ/RTI, JMP (\$xxFF) page-wrap, cycle-exact page-cross accounting
tests/6502/test_exhaustive.sage	20	ADC binary/decimal, SBC binary, branch cycle timing, page-cross penalties, JMP indirect page-wrap, IRQ/NMI B flags, PHP B flag, PLP bit 5 normalization, NMI priority
tests/compiler/test_asm6502.sage	6502	assembler encodes real 6502, labels resolve, program runs on the host emulator
tests/compiler/test_backend.sage	20	compiled BASIC output equality (arithmetic, strings, GOTO/IF, comparisons, div-0)
tests/boot/test_boot.sage	6	power-on banner + prompt
tests/boot/test_uart.sage	8	UART device RX/TX/status + echo-terminal
tests/boot/test_monitor.sage	10	monitor session (help/poke/peek/dump/regs/run/reset/unknown)

module	checks	what it proves
tests/basic/test_basic.sage	49	Applesoft arithmetic, strings, functions, control flow, errors
tests/display/test_spi.sage	9	SPI framing, CS, loopback, counters
tests/display/test_display.sage	29	OLED decode, windows, pixels/lines/text, 6502-driven
tests/storage/test_flash.sage	10	flash IDs, WEL, program/read, sector erase, 6502-driven
tests/storage/test_fs.sage	26	SAGEFS v2 round-trips, limits, persistence, BASIC save/load
tests/machine/test_speaker.sage	12	speaker model + BASIC/6502 driving
tests/machine/test_os.sage	24	the definition-of-done session
tests/machine/test_apple.sage	32	DOS 3.3 verbs, file types, monitor shell, CALL -151
Total	285	

How suites assert

Each test file defines a local `check(cond, msg)` (`test_cpu` uses manual counters) and ends with:

```
Results: N passed, 0 failed
ALL OK
```

Two suites (`test_os`, `test_monitor`) boot a full machine via `sageapple/machine.sage` and compare terminal transcripts; the compiler suites load produced binaries into the emulator and execute them; the storage suites instantiate the 64 KB chip model and operate the full 256-block filesystem.

The oracle / equivalence model

Beyond these checks, the strongest validation is the **host equivalence oracle** (see `docs/avr.md` and `docs/tools.md`): `tools/rom_gen.sage` records the reference session; `make host-test` replays it byte-for-byte against the compiled C core.

Running everything

```
for t in tests/**/*.sage; do sage $t; done
```

Nothing needs an emulator binary or the board: the machine itself runs in the interpreter. # Host tools — `tools/`

SageLang host-side utilities that build, flash, and validate artifacts. All run with the `sage` interpreter from the repo root.

tool	purpose
<code>tools/avr_boot.sage</code>	Sage → AVR opcodes → Intel HEX boot image
<code>tools/rom_builder.sage</code>	Intel HEX record emitter for 6502 ROMs
<code>tools/rom_gen.sage</code>	builds the monitor ROM + oracle transcript
<code>tools/gen_table.sage</code>	regenerates the C opcode table from the core
<code>tools/hex_dump.sage</code>	dumps <code>build/boot.bin</code> for inspection
<code>tools/applecon.sage</code>	artistic TUI + shell to connect to SageApple boards

`avr_boot.sage`

- AVR instruction encoders (`enc_ldi`, `enc_out`, `enc_rjmp`, `enc_in`) proven correct against the ATmega328P;
- a `emit_hex()` Intel-HEX record writer (checksums included);
- `build_boot()` emits a standalone AVR UART-init + TX blob (UDR0 written with 'H', then a self-loop) and writes `build/boot.hex`.

This was milestone-4 proof that **SageLang** → **AVR opcodes** → **Intel HEX** → **flash** works.

`rom_builder.sage`

`emit_8_hex(bytes, base)` — records of 16 bytes with an EOF record — shared by the ROM-generating tools.

`rom_gen.sage`

The canonical **oracle** generator. It:

1. assembles `sageapple/monitor.sage` into an 8 KB window (normally `$E000-$FFFF`); writes `avr/rom.inc` (PROGMEM array) and `avr/rom_host.c` (host array);

2. builds a 32 KB ROM (monitor at \$E000, reset vector \$E000);
3. **drives a real machine session** — help, regs, peek 0000, poke 0000 42, peek 0000, dump 0030 0030, xyz, run, reset, help — with a 60,000 step budget per command and writes the verbatim UART transcript to `avr/host_expected.txt`.

That transcript is the byte-exact equivalence target for the AVR port (`make host-test`), exactly as the on-chip monitor must reproduce it.

gen_table.sage

Reads `sage6502.cpu._OPCODES` (the canonical 256-entry `[id, mode]` table) and rewrites the `OP[256]` initializer inside `avr/sage6502.c`. Rerun it (from the repo root):

```
sage tools/gen_table.sage
```

and the C port's opcode table is rewritten to agree with the Sage core — the fix for the drift that silently broke the on-chip dispatch.

hex_dump.sage

Small helper that reads `build/boot.bin` and prints 16 bytes per line with offsets, for eyeballing generated binaries.

applecon.sage

The host-side controller for a rack of SageApple boards. It opens with an artistic TUI animation (banner + board glyph), then drops into an interactive `sage>` shell.

```
sage-c tools/applecon.sage # from the repo root
```

All three boards are wired to the OrangePi and reached over SSH:

con	board	serial port
0	og Uno R3	/dev/ttyUSB0
1	Nano R3	/dev/ttyUSB1
2	2nd Uno R3	/dev/ttyUSB2 (needs <code>cdc_acm</code>)

Commands:

command	action
<code>sage> con 0</code>	SSH to the OrangePi and connect to the og Uno R3
<code>sage> con 1</code>	SSH to the OrangePi and connect to the Nano R3
<code>sage> con 2</code>	SSH to the OrangePi and connect to the 2nd Uno R3

command	action
sage> status	probe each port and report which boards are present
sage> help	show the command list
sage> exit	leave AppleCon

Connection is handed off to **screen** for an interactive serial terminal (exits back to the **sage>** prompt with **C-a d/C-a k**). Stale detached screen sessions are automatically killed before each connect.

Note: run AppleCon with the C build (**sage-c**). The self-hosted RISC-V **sage** interpreter is unstable with the animation/shell loop.